

ASSIGNMENT 3

Each exercise is independent. Start from the provided code. The program structure is already given to you. Your task is to complete the parts marked `TODO` and then use the functions you implemented.

Exercise 1 Setlist manager

You should complete this exercise in <https://godbolt.org/z/Tf18PMv5j>.

You will complete a small command-line program that manages a concert setlist. Each song is represented by its length and stored as an integer in a `std::vector`.

The complete program structure is provided in the godbolt url. You will complete the functions marked `TODO`.

Step 1 — Complete the helper functions

Complete the functions marked with a `TODO` according to the following descriptions:

- `add_song(int duration)`: Adds a song to the setlist. A song with a duration of 0 or less is invalid and should not be added.
- `total_duration()`: Calculates the total duration of all songs currently in the setlist.
- `longest()`: Finds the duration of the longest song in the setlist. If there are no songs, return 0.
- `classify(int duration)`: Determines the tempo category of a song based on its duration
- `print_tempo(Tempo t)`: Converts a Tempo value into the corresponding text (slow, medium, or fast) and prints it. Use a switch.
- `print_setlist()`: Displays every song with its position and duration, then displays the total duration. Use the `total_duration()` function you wrote earlier.

Step 2 — Test your functions

Before continuing, uncommment the call to `test_functions` in `main`. Run the program and check that the output is:

```
song 1: 210s
song 2: 185s
```

```
song 3: 340s
total: 735s
longest: 340s
fast
```

Then comment it again.

Step 3 — Complete the command dispatch

Complete the TODOs in the main function. Use the functions you completed to do so.

Step 4 — Check your result

Test the following commands:

```
1 ./setlist add 240
```

Expected output:

```
setlist: 1 songs
```

```
1 ./setlist build 210 185 340 95 270
```

Expected output:

```
song 1: 210s
song 2: 185s
song 3: 340s
song 4: 95s
song 5: 270s
total: 1100s
```

Finally:

```
1 ./setlist analyse 210 185 340 95 270
```

Expected output:

```
total: 1100s
longest: 340s
fast
```

Also check the invalid case:

```
1 ./setlist add -5
```

Expected output:

```
invalid duration
setlist: 0 songs
```

Exercise 2 RGB image processor

This exercise should be completed with this: <https://godbolt.org/z/Eo4s4GM33>.

In this exercise, you will complete functions that manipulate a small RGB image. The images can then be visualised here: https://grybouilli.github.io/extra/img_visualizer.html. To give you a taste of the result, you can try to copy-paste the output of the program provided in the link in the visualiser and enjoy!

The image type and the images themselves are already provided. You only need to complete the functions marked `TODO`.

Step 1 — Understand the image type

There are three variables already defined that are images:

- `cat`
- `dog`
- `heart`

The first index selects the colour channel:

```
1 cat[0]    // red channel
2 cat[1]    // green channel
3 cat[2]    // blue channel
```

```
4
5 cat[0][12][14] // red channel of pixel (12, 14)
```

Step 2 — Complete `brightness`

Complete the function `brightness` that computes the brightness of a pixel in an image. Test it with:

```
1 std::cout << brightness(cat, 10, 10) << '\n';
```

Step 3 — Complete `channel_sum`

Complete the function `channel_sum` that sums all the pixel values in an image channel. Test your function by printing the three channel sums:

```
1 std::cout << channel_sum(cat, 0) << '\n';
2 std::cout << channel_sum(cat, 1) << '\n';
3 std::cout << channel_sum(cat, 2) << '\n';
```

Step 4 — Complete `brightest`

Complete the function `brightest`. Use the `brightness` function you wrote previously.

Step 5 — Complete `apply_offset`

The function `apply_offset` modifies one colour channel. Complete it. Now complete the second version so that it uses the first one:

```
1 Image apply_offset(Image img, int offset)
2 {
3     // TODO:
4     for(auto channel = 0; channel < 3; ++channel){
5
6         // Apply the offset to the red, green and blue
7         // channels.
8         //
9         // You should use the apply_offset function above
10        // rather than repeating its implementation.
11    }
```

```
12
13     return img;
14
15 }
```

Step 6 — Complete `process`

Most of the program is now finished.

Complete this function:

```
1 void process(const Image& image, const int offset)
2 {
3     // TODO:
4     // 1. Print the brightest pixel.
5     //
6     // 2. Apply the offset to image.
7     //
8     // 3. Print the resulting image using print_image().
9 }
```

This function should use the functions you implemented in the previous steps.

Step 7 — Use the functions in `main`

The `main` function is already provided, but you have to complete it.

The purpose of `main` is to select the correct image and call `process`. Fill the missing part.

Step 8 — Check your result

Try:

```
1 ./main cat 20
2 ./main dog -10
3 ./main heart 50
```

You can visualize the modified picture by copy-pasting the output of `print_image` in the left text pad of:

https://grybouilli.github.io/extra/img_visualizer.html